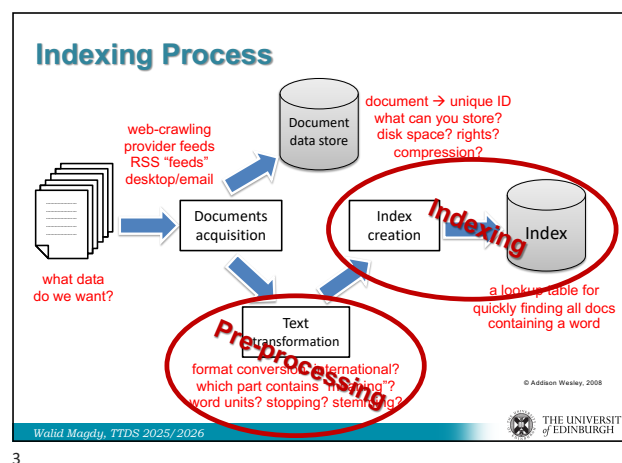


Indexing 1: 从扫全文到倒排索引

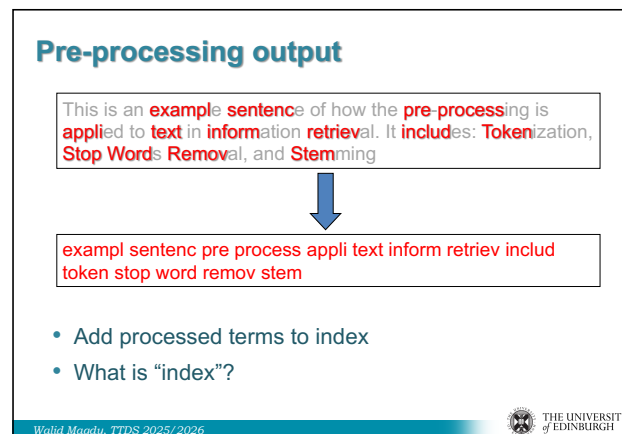
图文讲义版：用原 PPT 作为视觉锚点，按“概念故事线 + 直觉解释 + 考试速记”整理。

preprocessing 产出 term → linear scan 太慢 → document/term matrix → inverted index → postings 与 df/tf → Boolean query processing → phrase/proximity search

10/1/25



3



4

2

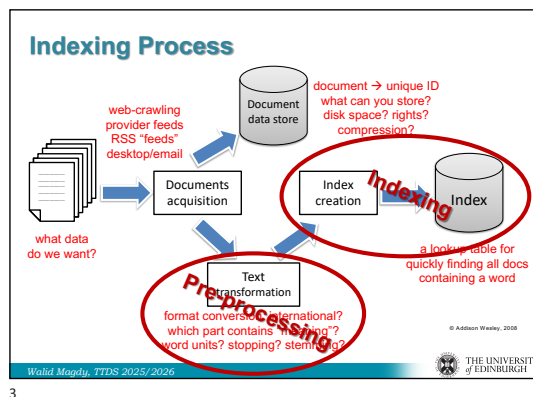
原 PPT 第 2 页

这节课一句话

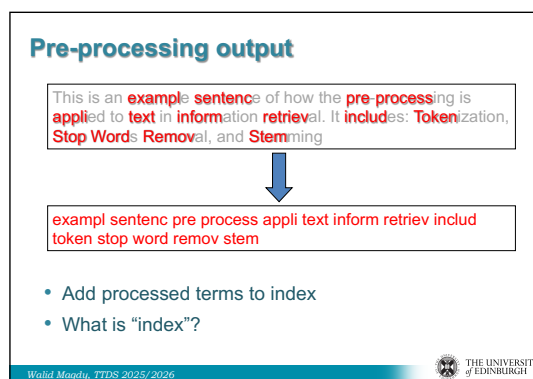
这节课把搜索引擎的“找词”问题，从笨拙的全文扫描，变成 term 到 postings list 的高速查表问题。

1. Index 是搜索引擎的“地图册”

10/1/25



3



4

2

原 PPT 第 2 页

人话直觉

没有索引的搜索像在整座城市挨家敲门；有索引之后，你先查地图，直接去相关街区。

精确定义 / 机制：文档经过 crawling、feeds 或本地 acquisition 进入 data store，再做 format conversion、tokenisation、stopping、stemming 等 preprocessing，最后把 processed terms 加入 index。index 本质上是 lookup table：给定一个 term，快速找到包含它的 documents 或位置，而不是重新扫描 collection。考试问法：若问 indexing process，要说清楚：documents acquisition/preprocessing 产出 terms，index creation 建立 term 到 document evidence 的映射。

2. 为什么 grep/find 不够用

10/1/25

Indexing

- Search engines vs PDF find or grep?
 - Infeasible to scan large collection of text for every "search"
 - Find section that has: "UK and Scotland and Money"?!
- Book Index
 - For each word, list of "relevant" pages
 - Find topic in sub-linear time
- IR Index:
 - Data structure for fast finding terms
 - Additional optimisations could be applied

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY of EDINBURGH

7

Document Vectors

- Represent documents as vectors
 - Vector → document, cell → term
 - Values: term frequency or binary (0/1)
 - All documents → collection matrix

	he	drink	ink	likes	pink	think	wink
D1: He likes to wink, he likes to drink	2	1	0	2	0	0	1
D2: He likes to drink, and drink, and drink	1	3	0	1	0	0	0
D3: The thing he likes to drink is ink	1	1	1	1	0	1	0
D4: The ink he likes to drink is pink	1	1	1	1	1	0	0
D5: He likes to wink, and drink pink ink	1	1	1	1	1	0	1

number of occurrence of a term in a document

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY of EDINBURGH

8

4

原 PPT 第 4 页

人话直觉

grep 像每次考试都把整本书从第一页读到最后；搜索引擎必须像翻书后索引一样，马上跳到相关页。

精确定义 / 机制： Sequential search 对每次 query 都线性扫描文本，collection 很大、query 很多时不可行。IR index 的作用是 sub-linear 地定位 term 所在文档，并可额外保存 frequency、positions 等信息支持更复杂检索。

考试问法：常考比较：PDF find/grep 是 linear scan；IR index 是为大规模、高频查询设计的数据结构。

3. Document vector 与 collection matrix

10/1/25

Collection Matrix

	Documents					
	Antony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	1	1	0	0	0	1
Brutus	1	1	0	1	0	0
Caesar	1	1	0	1	1	1
Calpurnia	0	1	0	0	0	0
Cleopatra	1	0	0	0	0	0
mercy	1	0	1	1	1	1
worse	1	0	1	1	1	0

1 if document contains term, 0 otherwise

Query: Brutus AND Caesar AND NOT Calpurnia
Apply on rows: 110100 AND 110111 AND !010000 = 100100

Walid Magdy, TTDS 2025/2026

11

Bigger collections?

- Consider $N = 1$ million documents, each with about 1000 words.
- $n = 1M \times 1K = 1B$ words
→ Heap's law → $v \approx 500K$
- Matrix size = 500K unique terms $\times$ 1M documents
= 0.5 trillion 0's and 1's entries!
- If all words appear in many documents
→ $\max\{\text{count}\{1's\}\} = N \times \text{doc. length} = 1B$
- Actually, from Zip's law → 250k terms appears once!
- Collection matrix is extremely **sparse**. (mostly 0's)

Walid Magdy, TTDS 2025/2026

12

6

原 PPT 第 6 页

人话直觉

把每篇文档想成超长购物清单：每个词是一格，格子里写这个词买了几次，也就是出现几次。

精确定义 / 机制：document vector 以 terms 为维度，cell value 可以是 binary 0/1 或 term frequency。所有 document vectors 叠起来形成 collection matrix。问题是 vocabulary 很大，而每篇文档只含少量 terms，所以矩阵绝大多数 entry 是 0。

考试问法：若问 collection matrix 为什么 sparse，要联系 vocabulary 大、单篇文档词项少，以及 Zipf/Heaps 导致大量 rare terms。

4. Inverted index: 把矩阵翻过来

10/1/25

Inverted Index

- Represent terms as vectors
 - Vector → term, cell → document
 - Transpose of the collection matrix
 - Vector: inverted list

	he	drink	ink	likes	pink	thing	wink	
	2	1	0	2	0	0	1	← D1: He likes to wink, he likes to drink
	1	3	0	1	0	0	0	← D2: He likes to drink, and drink, and drink
	1	1	1	1	0	1	0	← D3: The thing he likes to drink is ink
	1	1	1	1	1	0	0	← D4: The ink he likes to drink is pink
	1	1	1	1	1	0	1	← D5: He likes to wink, and drink pink ink

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY OF EDINBURGH

9

Boolean Search

- Boolean: exist / not-exist
- Multiword search: logical operators (AND, OR, NOT)
- Example
 - Collection: search Shakespeare's Collected Works
 - Boolean query: Brutus AND Caesar AND NOT Calpurnia
- Build a **Term-Document Incidence Matrix**
 - Which term appears in which document
 - Rows are terms
 - Columns are documents

Walid Magdy, TTDS 2025/2026

THE UNIVERSITY OF EDINBURGH

10

5

原 PPT 第 5 页

人话直觉

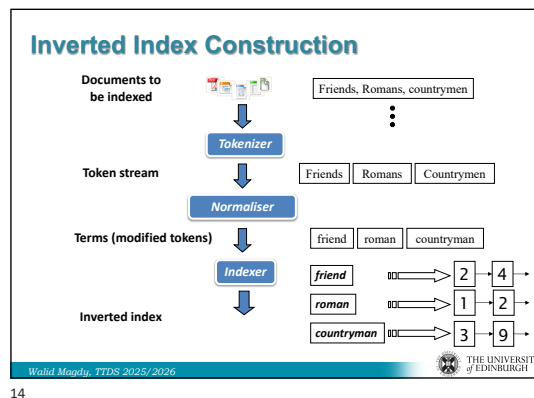
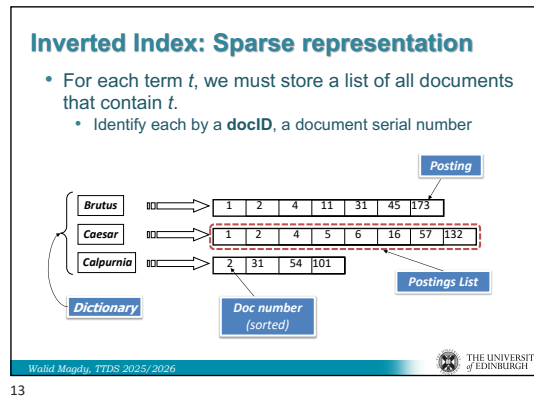
不是问“这篇文档有哪些词”，而是反过来问“这个词在哪些文档里”。搜索正好需要后者。

精确定义 / 机制：inverted index 用 term 作为入口，每个 term 对应一个 inverted list/postings list，列出包含该 term 的 documents。它等价于 collection matrix 的稀疏表示：只存非零的 term-document pairs，不存海量 0。

考试问法：定义 inverted index 时要包含 term → postings list，posting 通常至少有 docID，可扩展 tf 和 positions。

5. Dictionary、posting、postings list、df

10/1/25



7

原 PPT 第 7 页

人话直觉

dictionary 像电话簿目录，postings list 像某个名字下面的所有地址，df 是地址数量。

精确定义 / 机制： dictionary 保存 vocabulary terms 及 metadata，例如 document frequency df 和指向 postings list 的 pointer。posting 是 postings list 的一条记录，通常记录 docID； $df(t)$ 是包含 term t 的 document 数量。按 docID 排序后可以用 linear merge 高效做 AND/OR。

$$df(t) = |\{d : t \in d\}|$$

考试问法： 考试容易让你区分 df 和 posting：df 是数量，posting 是记录，postings list 是记录列表。

6. Index construction: 先排序, 再合并

10/1/25

Step 3: Posting

1. Multiple term entries in a single document are merged
2. Split into Dictionary and Postings
3. Doc. Frequency (df) information is added

Term	docID
ambitious	2
be	2
brutus	1
brutus	2
capitol	1
caesar	1
caesar	2
caesar	2
did	1
enact	1
hath	1
i	1
i	1
i	1
it	2
julius	1
killed	1
killed	1
let	2
me	1
noble	2
so	1
the	1
the	2
told	2
you	2
was	1
was	2
with	2

term	doc. freq.
ambitious	1
be	1
brutus	2
capitol	1
caesar	2
did	1
enact	1
hath	1
i	1
i	1
i	1
it	1
julius	1
killed	1
let	1
me	1
noble	1
so	1
the	2
told	1
you	1
was	2
with	1

→ postings lists

17

Inverted Index: matrix → postings

	he	drink	ink	likes	pink	think	wink
1	2	1	0	0	2	0	1
2	1	3	0	1	0	0	0
3	1	1	1	1	0	1	0
4	1	1	1	1	1	0	0
5	1	1	1	1	1	0	1

← D1: He likes to wink, he likes to drink
← D2: He likes to drink, and drink, and drink
← D3: The thing he likes to drink is ink
← D4: The ink he likes to drink is pink
← D5: He likes to wink, and drink pink ink

he → 1 2 3 4 5
drink → 1 2 3 4 5
ink → 3 4 5
pink → 4 5
thing → 3
wink → 1 5

18

9

原 PPT 第 9 页

人话直觉

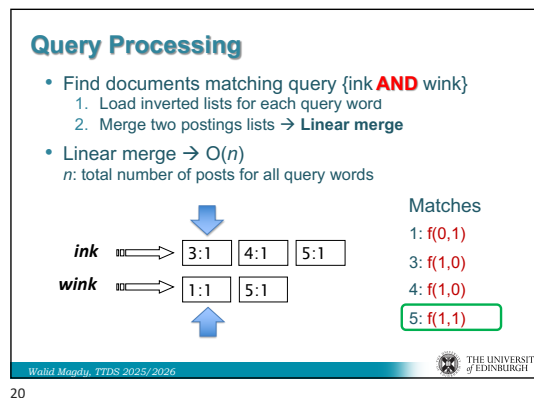
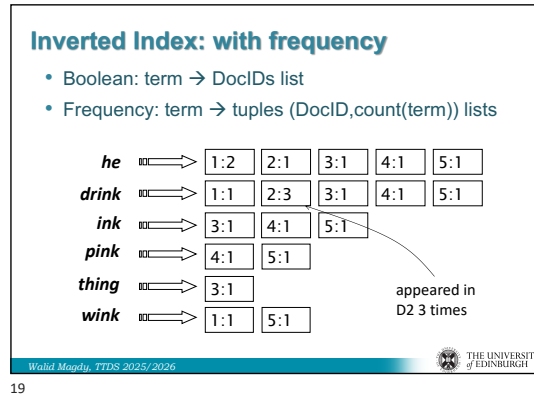
像把散落的快递单先按收件人排序, 再把同一人的包裹合成一条清单。

精确定义 / 机制: 构建 inverted index 的典型步骤是: 从每篇文档生成 (term,docID) 序列; 按 term 再按 docID 排序; 合并同一 term/document 的重复出现; 最后生成 dictionary、df 和 postings。若保留 count, 就得到 frequency inverted index。

考试问法: 若问 construction pipeline, 按 tokenize/normalise → term-doc pairs → sort → merge → dictionary/postings 作答。

7. Boolean query processing: 集合运算

10/1/25



10

原 PPT 第 10 页

人话直觉

AND 是两张名单取交集，OR 是合并名单，NOT 是从全集里划掉名单。

精确定义 / 机制： Boolean retrieval 对 query terms 的 postings lists 做集合运算。AND 常用 intersection，OR 用 union，NOT 用 complement。由于 postings 通常按 docID 排序，两个 list 的 intersection 可用 linear merge 完成，时间与 list 长度之和成正比。

$$T_{AND} = O(|P_{ink}| + |P_{wink}|)$$

考试问法： 若问 query {ink AND wink}，要说加载两个 postings lists，然后做 intersection/linear merge。

8. Phrase search: AND 还不够

10/1/25

Phrase Search

- Find documents matching query "pink ink"
 - Find document containing both words
 - Both words has to be a phrase
- Bi-gram Index:
He likes to wink, and drink pink ink → Convert to bigrams
He_likes likes_to to_wink wink_and and_drink drink_pink pink_ink
- Bi-gram Index, issues:
 - Fast, but index size will explode!
 - What about trigram phrases?
 - What about proximity? "ink is pink"

Walid Magdy, TTDS 2025/2026

21

Proximity Index

- Terms positions is embedded to the inv. Index
 - Called proximity/positional index
 - Enables phrase and proximity search
 - Tuples (DocID, term position)

he ⇒ 1:2 2:1 3:1 4:1 5:1
drink ⇒ 1:1 2:3 3:1 4:1 5:1

D1: He likes to wink, he likes to drink
D2: He likes to drink, and drink and drink
D3: The thing he likes to drink is ink
D4: The ink he likes to drink is pink
D5: He likes to wink, and drink pink ink

he ⇒ 1,1 1,5 2,1 3,3 4,3 5,1
drink ⇒ 1,8 2,4 2,6 2,8 3,6 4,5 5,6

Walid Magdy, TTDS 2025/2026

22

11

原 PPT 第 11 页

人话直觉

“pink”和“ink”都出现，只说明两个人在同一个房间；phrase search 要求他们手拉手按顺序站着。

精确定义 / 机制：phrase query 不仅要求 terms 同时出现在文档中，还要求顺序和相邻关系满足短语。bi-gram index 可把相邻词对如 $pink_i ink$ 当作 index term，但会增大 index，并不自然支持任意长度 phrase 或 proximity query。

考试问法：常考 phrase search vs Boolean AND：AND 只要求共现；phrase 还要求 positions 满足顺序相邻。

9. Positional/proximity index: 保存词的位置

10/1/25

Proximity search: data structure

- Possible data structure:
 <term: df;
 DocNo: pos1, pos2, pos3
 DocNo: pos1, pos2, pos3
 >
- Example:
 <scotland: 25;
 94: 212
 351: 23,34,1354,1779,1838
 370: 981
 3330: 115
 3334: 121,316
 3532: 59,111,162,265 ...>

Walid Magdy, TTDS 2025/2026

25

Practical

Walid Magdy, TTDS 2025/2026

26

13

原 PPT 第 13 页

人话直觉

如果只知道词在哪篇文章里，你只能知道“同场”；如果知道位置，你才能判断“距离多近”。

精确定义 / 机制：positional index 在 posting 中保存 term positions，例如 term: df; DocNo: pos1,pos2,pos3。phrase search 可检查相邻位置，proximity search 可检查两个 term 的距离是否不超过窗口 n 。代价是 index size 增大，但换来短语和邻近查询能力。

$$|pos(t_1, d) - pos(t_2, d)| \leq n$$

考试问法：若问 proximity search，答案要包含两步：先 AND 找候选文档，再检查 positions 距离。

考试速记版

- Index 的核心作用：避免每次 query 线性扫描整个 collection。

- Inverted index 是 term $\rightarrow$ postings list; posting 至少含 docID, 可扩展 tf/positions。
- Collection matrix 很 sparse, 所以 inverted index 只存非零 term-document evidence。
- Boolean query processing 本质是 postings lists 的集合运算。
- Phrase/proximity search 需要 positional evidence; Boolean AND 本身不够。
- Index construction 关键词: term-doc pairs、sort、merge、dictionary、postings、df。

一段稳妥考场回答

Inverted index 是 IR 系统中最核心的索引结构, 它把每个 term 映射到包含该 term 的 postings list。与完整 term-document matrix 相比, 它只保存非零出现记录, 因此是稀疏且高效的表示。构建时通常先从预处理后的文档生成 (term, docID) 序列, 排序并合并重复项, 得到 dictionary、document frequency 和 postings。查询时, Boolean AND/OR/NOT 可转化为 postings lists 的交集、并集和补集; 若要支持 phrase 或 proximity search, 则需要在 posting 中额外保存 term positions。